REAL CLIENT STORY + OPEN-SOURCE FRAMEWORK

A Client Was Losing Hundreds of Thousands Because They Couldn't Answer Every Complaint on Time

How we cut missed deadlines by 35% with a hybrid AI system — and the framework you can use too.

Built on a similar Kaggle dataset · Shared as an example · Full code on GitHub

35% FEWER BREACHES	**$574K** ANNUAL VALUE
69x ROI	**72%** HUMAN APPROVAL

The Problem — In Plain English

When a customer submits a support ticket — a complaint, a question, a request for help — there's usually a **promise attached.**

"We'll get back to you within 24 hours." "We'll resolve this within 48 hours."

This promise is called an **SLA — a Service Level Agreement.** Think of it like a delivery guarantee. If you order something with 2-day shipping and it arrives in 5 days, that's a broken promise. Same thing here.

> *My client was breaking that promise on nearly half of their tickets. **48%.** Every other customer who reached out waited longer than they were told.*

When you break that promise, real things happen: contract penalties, refund obligations, customers who leave and never come back.

The cost was adding up to **hundreds of thousands a year** — not from one big failure, but from thousands of small ones. Tickets that slipped through the cracks. Complaints sitting in the wrong queue. Issues nobody noticed were about to expire.

Their instinct? **"Let's use AI. Let an AI agent handle it."**

That would have been the wrong move. Here's why.

Why "Just Let AI Handle It" Fails Here

Would you let a brand-new employee make decisions about your biggest clients on day one — no playbook, no supervision? That's what a fully autonomous AI agent does.

⚡ Consistency

The AI looks at the same ticket twice. Run 1: "urgent, escalate immediately." Run 2: "medium priority, just reassign." **Same ticket, different answer.** For operational decisions, you need identical outputs every time. Otherwise, nobody trusts it.

⚡ Speed

A rule-based system answers in **2 seconds.** An AI agent that "thinks" takes **8-15 seconds.** Across hundreds of tickets daily, those seconds compound into hours of delay.

⚡ Cost

Every time an AI agent "thinks," it makes **3–7 API calls** per ticket. A pipeline makes **1.** At 20,000 tickets, that's the difference between a system that pays for itself and one that's too expensive to run.

Agents are not "better AI." They're a different tool for a different job.

The Solution: 4-Layer Architecture

Use **rules where you need reliability.** Use **AI where you need judgment.** Both, in the right places.

! The Problem

20,000 tickets — 48% breaching SLA deadlines. Half your promises? Broken.

▼

1 Early Warning System

When a ticket uses 70% of its allotted time, flag it. Like a yellow light — still time, but act now. Caught 1,474 tickets.

▼

2 Hybrid Decision Engine

Two brains working together — details on next slide.

60%
Rule-Based Math
Time left? Priority? Reassignments? Same answer every time.

+

40%
LLM Judgment
Customer frustrated? Simple or complex issue? Context rules miss.

▼

3 Bounded Actions (The Guardrails)

AI picks from exactly 9 predefined actions. Cannot invent new ones. Every recommendation → human approval.

▼

✓ Result: 35% Fewer Breaches

48% → 31%. Resolution 12 hours faster. $574K annual value.

The 9 Actions — Nothing More, Nothing Less

The single most important design decision: **the AI can only recommend from a fixed menu.** No creativity. No improvisation. Like giving someone a clear playbook instead of saying "figure it out."

⬆ Escalate Priority	□ Reassign Ticket	□ Contact Customer
□ Set Reminder	□ Add Resources	□ Update Status
□ Request Collab	□ Flag for Review	□ Add Internal Note

Why boundaries beat better prompts:

This one constraint moved human approval from **55%** to **72%.** When something goes wrong, you can trace exactly what happened: "The AI recommended escalation because the time-remaining score was 0.23 and the LLM detected frustration." That's debuggable. That builds trust.

The Results — Before & After

SLA BREACH RATE

Before 48.2%

After 31.4%

RESOLUTION TIME

Before 40 hrs

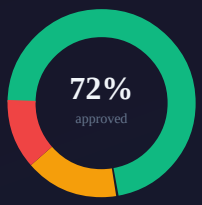
After 28 hrs

CUSTOMER SATISFACTION

Before 3.8 / 5

After 4.2 / 5

HUMAN APPROVAL



- Approved 72%
- Modified 16%
- Rejected 12%

$492K
SLA Prevention

$90K
Productivity Gains

-$8.4K
System Cost

SAVE THIS

When to Use Pipelines vs. Agents

The checklist I use before building any AI feature. It saves months of building the wrong thing.

Use a Pipeline When...

- → Output must be the same every time
- → Speed matters (under 3 seconds)
- → You need to trace what went wrong
- → Steps are known and don't change
- → Compliance requires a paper trail
- → Cost per call matters at scale

Use an Agent When...

- → Reasoning across multiple sources
- → 3+ steps with branching logic
- → Involves negotiation or optimization
- → The path isn't known in advance
- → User interaction mid-task helps
- → The reasoning IS the value

Most real products need BOTH.

The skill is knowing where one ends and the other begins.

5 Lessons That Apply to Any AI Product

LESSON 01

Start deterministic. Add agents later.

It's easy to add intelligence to a working system. It's brutally hard to add reliability to a flaky one.

LESSON 02

Agents are not "better AI."

Using an agent for classification is like using a search engine to do arithmetic — possible, but why?

LESSON 03

The hybrid wins. Every time.

60% rules + 40% AI wasn't a compromise. It was the winning formula.

LESSON 04

Humans stay in the loop.

We went from 55% → 72% approval not by improving prompts, but by giving humans clear, bounded recommendations.

LESSON 05

Set boundaries before you deploy.

The 9-action limit wasn't a constraint — it was the reason the system earned trust. Constraints create confidence.

OPEN SOURCE

The Full Logic Is On GitHub

I found a similar dataset on Kaggle and rebuilt the full architecture — code, data, logic, results — so you can explore the approach and adapt it.

View on GitHub →

Found this useful?

Save it · Share it · Follow for more

@lanabuildsai